

Contents

Reactor Yield Surrogate	1
1. The chemistry, and what makes it hard	1
2. What the data says before any modelling	3
3. The model: recover the reactor, do not approximate it	5
4. Validation: how we avoided fooling ourselves on 150 rows	9
5. Model selection: what we tested and rejected	13
6. What the recovered parameters mean	16
7. Predictions and submission	20
8. Robustness, extrapolation and scaling	22

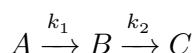
Reactor Yield Surrogate

Team Claude ke Chhatore • Predictive Modeling Optimization Challenge

We recovered the reactor's governing equations instead of fitting a general-purpose regressor to 150 rows. Five operating knobs in, yield of product B out.

Every fitted parameter is an activation energy, a heat of reaction or a heat-transfer coefficient, so a process engineer can check it against known kinetics. The model is also about **4.5× more accurate** than the best tree ensemble we could build.

1. The chemistry, and what makes it hard



Two first-order reactions in series. B is the product **and** the feedstock for the waste reaction, so it is created and destroyed at once. Both rate constants are Arrhenius, so heat accelerates both, but not equally: if $E_2 > E_1$, heating destroys B faster than it forms it and selectivity collapses.

Knob	Set by
Temperature	inlet_temperature_K, jacket_temperature_K
Residence time $\tau = L/Q$	length_m, flow_rate_L_min

The result is a **ridge**. Too cold or too fast and A never converts; too hot or too slow and B is destroyed. The optimum is a narrow band between the two.

This notebook is self-contained. The integrator, the fit, the validation and the submission writer are all defined and executed below, with nothing essential imported from a local package, so it runs anywhere the two CSVs and standard scientific Python are present. Cached results are used only where a step is slow, and each one is recomputed and asserted rather than trusted.

```
import json
from pathlib import Path
from dataclasses import dataclass

import numpy as np, pandas as pd
import matplotlib.pyplot as plt

# One restrained figure style for the whole notebook.
plt.rcParams.update({"figure.dpi": 110, "axes.grid": True, "grid.alpha": 0.25,
                    "axes.spines.top": False, "axes.spines.right": False,
                    "font.size": 10, "axes.titlesize": 11, "axes.titleweight": "bold",
                    "legend.frameon": False})
```

```

# Locate the project regardless of where the notebook is launched from.
ROOT = next(p for p in [Path.cwd(), *Path.cwd().parents]
            if (p / "data" / "train_dataset.csv").exists()
            or (p / "train_dataset.csv").exists())
DATA = ROOT / "data" if (ROOT / "data" / "train_dataset.csv").exists() else ROOT
ART = ROOT / "artifacts"

TARGET = "overall_yield"
R_GAS = 8.314 # J/(mol K)
T_REF = 430.0 # K -- Arrhenius reference, ~ the data mean
Q_REF = 40.0 # L/min
TRIAL_E = (60_000.0, 100_000.0, 160_000.0)
DEFAULT_STEPS, SEARCH_STEPS, SUBMIT_STEPS = 512, 256, 2048
PARAM_NAMES = ("ln_k1_ref", "E1_kJ", "ln_k2_ref", "E2_kJ", "a1", "a2", "U", "n_flow")

train = pd.read_csv(DATA / "train_dataset.csv")
test = pd.read_csv(DATA / "test_dataset.csv")
y = train[TARGET].to_numpy()

def rmse(a, b):
    return float(np.sqrt(np.mean((np.asarray(a) - np.asarray(b)) ** 2)))

print(train.shape, test.shape, "| missing values:", train.isna().sum().sum())
train.head()

```

```
(150, 6) (50, 5) | missing values: 0
```

	flow_rate_L_min	concentration_mol_L	inlet_temperature_K	length_m \
0	33.09	3.68	357.75	19.87
1	76.30	1.34	429.70	14.84
2	59.90	1.01	431.10	11.76
3	49.90	2.21	445.61	22.85
4	16.70	3.95	458.91	4.56

	jacket_temperature_K	overall_yield
0	383.79	63.024
1	405.72	86.611
2	385.40	86.347
3	367.74	92.175
4	374.13	82.211

```

@dataclass(frozen=True)
class ReactorParams:
    ln_k1_ref: float
    E1_kJ: float
    ln_k2_ref: float
    E2_kJ: float
    a1: float
    a2: float
    U: float
    n_flow: float = 0.0

    @classmethod
    def from_vector(cls, x) -> "ReactorParams":
        # Vectors saved before n_flow existed are 7 Long; treat them as n_flow=0.
        vals = [float(v) for v in x]

```

```

        if len(vals) == len(PARAM_NAMES) - 1:
            vals.append(0.0)
        return cls(*vals)

    def to_vector(self) -> np.ndarray:
        return np.array([getattr(self, n) for n in PARAM_NAMES], dtype=float)

    def as_dict(self) -> dict[str, float]:
        return {n: getattr(self, n) for n in PARAM_NAMES}

def add_physics_features(df):
    """Reaction-engineering derived features (not polynomial combinatorics)."""
    out = df.copy()
    tau = df["length_m"] / df["flow_rate_L_min"]
    out["tau"], out["log_tau"] = tau, np.log(tau)
    out["T_avg"] = (df["inlet_temperature_K"] + df["jacket_temperature_K"]) / 2.0
    out["delta_T"] = df["jacket_temperature_K"] - df["inlet_temperature_K"]
    out["inv_T_avg"] = 1.0 / out["T_avg"]
    for e in TRIAL_E:
        # Log-Damkohler numbers
        out[f"ln_Da_{int(e/1000)}k"] = out["log_tau"] - e / (R_GAS * out["T_avg"])
    return out

FEATURE_COLUMNS = ["flow_rate_L_min", "concentration_mol_L", "inlet_temperature_K",
                    "length_m", "jacket_temperature_K", "tau", "log_tau", "T_avg",
                    "delta_T", "inv_T_avg"] + [f"ln_Da_{int(e/1000)}k" for e in TRIAL_E]

def ode_inputs(df):
    """The four arrays the reactor integrator needs, one element per row."""
    return {"CA0": df["concentration_mol_L"].to_numpy(float),
            "T_in": df["inlet_temperature_K"].to_numpy(float),
            "T_jacket": df["jacket_temperature_K"].to_numpy(float),
            "tau": (df["length_m"] / df["flow_rate_L_min"]).to_numpy(float),
            "Q": df["flow_rate_L_min"].to_numpy(float)}

```

2. What the data says before any modelling

Three facts shaped every decision that followed.

```

f = add_physics_features(train)
print(f"exact zeros:      {(y == 0).mean():.1%}    ({(y < 1).mean():.0%} below 1.0)")
print(f"above 90:        {(y > 90).mean():.1%}")
print()
for c in ["jacket_temperature_K", "inlet_temperature_K", "concentration_mol_L",
          "flow_rate_L_min", "length_m"]:
    print(f"  corr({c:22s}, yield) = {np.corrcoef(train[c], y)[0,1]:+.3f}")
print(f"  corr({'log_tau':22s}, yield) = {np.corrcoef(f.log_tau, y)[0,1]:+.3f}")

exact zeros:      24.7%    (38% below 1.0)
above 90:         12.0%

```

```

corr(jacket_temperature_K , yield) = -0.498
corr(inlet_temperature_K  , yield) = -0.405
corr(concentration_mol_L   , yield) = +0.009
corr(flow_rate_L_min      , yield) = +0.038

```

```

corr(length_m, yield) = +0.080
corr(log_tau, yield) = +0.061
fig, ax = plt.subplots(1, 3, figsize=(14.5, 4.1))

# (a) the target itself: the exact-zero spike and the second mode at the top
n_zero = int((y == 0).sum())
ax[0].hist(y, bins=np.linspace(0, 100, 41), color="#4c72b0",
           edgecolor="white", linewidth=0.4)
ax[0].annotate(f"exactly zero\n{n_zero} rows ({(y == 0).mean():.1%})",
               xy=(1.5, n_zero), xytext=(22, n_zero * 0.82), color="#c44e52",
               arrowprops=dict(arrowstyle="->", color="#c44e52", lw=1.2))
ax[0].set_xlabel("overall_yield (%)"); ax[0].set_ylabel("training rows")
ax[0].set_title("(a) zero-inflated and bimodal")

# (b) the same six correlations printed above, as a chart
cols = ["jacket_temperature_K", "inlet_temperature_K", "length_m",
        "flow_rate_L_min", "concentration_mol_L"]
short = ["jacket T (K)", "inlet T (K)", "length (m)",
         "flow rate (L/min)", "conc (mol/L)", "log tau"]
r = [np.corrcoef(train[c], y)[0, 1] for c in cols] + [np.corrcoef(f.log_tau, y)[0, 1]]
o = np.argsort(r)
ax[1].barh([short[i] for i in o], [r[i] for i in o],
           color=["#c44e52" if r[i] < 0 else "#55a868" for i in o])
for i, k in enumerate(o):
    ax[1].text(r[k] + (0.02 if r[k] >= 0 else -0.02), i, f"{r[k]:+.3f}",
              va="center", ha="left" if r[k] >= 0 else "right", fontsize=9)
ax[1].axvline(0, color="k", lw=0.8); ax[1].set_xlim(-0.72, 0.32)
ax[1].set_xlabel("Pearson r with overall_yield")
ax[1].set_title("(b) temperature dominates, concentration does not")

# (c) the interior optimum that makes corr(log_tau, yield) vanish. The overlay is a
# binned median of the plotted points, not a fit.
s = ax[2].scatter(f.tau, y, c=f.T_avg, cmap="coolwarm", s=26,
                 edgecolor="k", linewidth=0.3, zorder=2)
edges = np.quantile(f.log_tau, np.linspace(0, 1, 7))
mid = [np.exp((edges[i] + edges[i+1]) / 2) for i in range(6)]
med = [np.median(y[(f.log_tau >= edges[i]) &
                  (f.log_tau <= edges[i+1] if i == 5 else f.log_tau < edges[i+1])])
       for i in range(6)]
ax[2].plot(mid, med, "-o", color="#333333", lw=1.6, ms=4, zorder=3,
          label="median per tau sextile")
ax[2].legend(loc="upper left", fontsize=9)
ax[2].set_xscale("log")
ax[2].set_xlabel("residence time tau = L/Q (m·min/L)")
ax[2].set_ylabel("overall_yield (%)")
ax[2].set_title("(c) yield rises then collapses with tau")
plt.colorbar(s, ax=ax[2], label="mean T (K)")

plt.tight_layout(); plt.show()

```

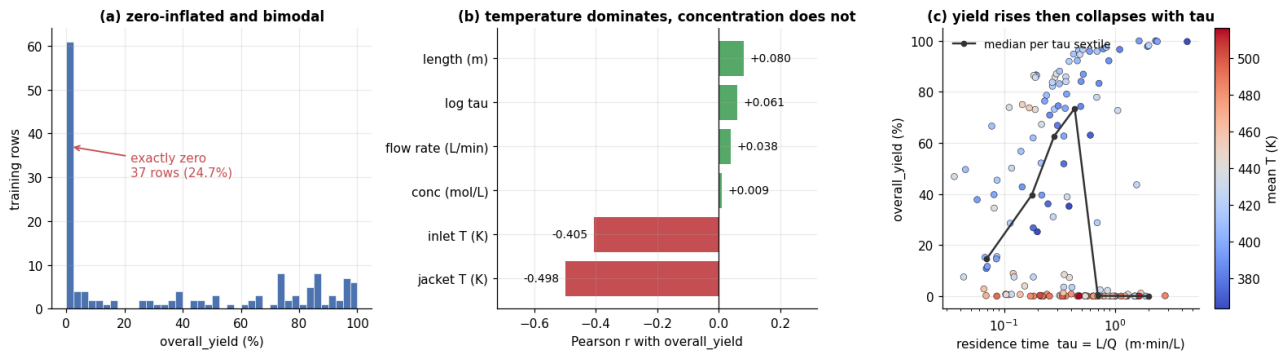


Figure 1. (a) a quarter of the rows are exactly zero, which rules out any log target transform. (b) jacket temperature is the dominant linear driver while inlet concentration is essentially uncorrelated. (c) the binned median of the plotted points rises then collapses: this is the interior optimum in residence time, and it is what cancels the linear correlation with log tau.

(a) The target is zero-inflated and bimodal. A quarter of the rows are *exactly* zero: a dead reactor. No log or Box-Cox transform of the target is admissible, since none can represent an exact zero.

(b) Temperature dominates and the sign is negative. Hotter reactor, lower yield. That is direct evidence for $E_2 > E_1$ before anything is fitted.

(c) $\text{corr}(\log_tau, \text{yield}) \approx 0.06$ does not mean residence time is unimportant. It is the signature of an **interior optimum**: low τ leaves A unconverted, high τ destroys B, so both tails are low-yield and the linear correlation cancels. τ is the single most important derived quantity in the problem.

3. The model: recover the reactor, do not approximate it

We integrate the governing system along the reactor axis:

$$\frac{dC_A}{dz} = -k_1 C_A, \quad \frac{dC_B}{dz} = k_1 C_A - k_2 C_B, \quad \frac{dT}{dz} = a_1 k_1 C_A + a_2 k_2 C_B + U(T_{jacket} - T)$$

with $C_A(0) = C_{A0}$, $C_B(0) = 0$, $T(0) = T_{inlet}$, integrated to $\tau = L/Q$; the yield is $100 C_B(\tau)/C_{A0}$. Seven fitted parameters: $\ln k_1^{ref}$, E_1 , $\ln k_2^{ref}$, E_2 , a_1 , a_2 , U .

Three implementation decisions did the real work.

1. **Arrhenius reparameterized about $T_{ref} = 430 \text{ K}$** , as $k = e^{\ln k_{ref}} \exp[-\frac{E}{R}(\frac{1}{T} - \frac{1}{T_{ref}})]$. Fitting $\ln A$ and E directly correlates them above 0.999 and turns the objective into a long narrow valley, which is the usual reason this fit is reported as slow to converge.
2. **All 150 rows integrate simultaneously** under operator splitting: rates frozen per substep, the mass balance advanced *analytically* (exact solution of the linear series reaction), the energy balance advanced with the exact linear solution. Both halves are unconditionally stable, so no step size can produce negative concentrations. One 150-row evaluation costs about 22 ms, which is what made global multistart search affordable.
3. **Parameter sets whose integration has not converged are rejected.** Without this the optimizer minimizes *integration error* rather than data error: an early fit produced parameters whose predictions moved 92 yield-points when the substep count was raised.

3a. The integrator, in full

The complete solver, with no library ODE call in the inner loop. It is reproduced verbatim from the module we test against SciPy, so what you read is what produced every number below.

```
def _rate(ln_k_ref: float, E_kJ: float, T: np.ndarray) -> np.ndarray:
    """Arrhenius rate constant, reparameterized about T_REF."""
    return np.exp(ln_k_ref - (E_kJ * 1000.0 / R_GAS) * (1.0 / T - 1.0 / T_REF))
```

```

def _series_step(xA, xB, k1, k2, dz):
    """Exact solution of the linear series reaction over dz at frozen k1, k2.

    
$$x_A' = x_A \exp(-k_1 dz)$$


$$x_B' = x_B \exp(-k_2 dz) + x_A * k_1 / (k_2 - k_1) * (\exp(-k_1 dz) - \exp(-k_2 dz))$$


    This is what makes the scheme unconditionally stable: however large  $k dz$ 
    gets, the exponentials decay to zero instead of oscillating. A fixed-step
    RK4 blows up here once  $k dz$  exceeds  $\sim 2.8$ .
    """
    e1 = np.exp(-k1 * dz)
    e2 = np.exp(-k2 * dz)
    d = k2 - k1
    # Removable singularity at k1 == k2: fall back to the confluent limit
    #  $x_B' = x_B e1 + x_A k_1 dz e1$ .
    near = np.abs(d * dz) < 1e-8
    d_safe = np.where(near, 1.0, d)
    gain = np.where(near, k1 * dz * e1, k1 / d_safe * (e1 - e2))
    xA_new = xA * e1
    xB_new = xB * e2 + xA * gain
    return xA_new, xB_new

def _temperature_step(T, T_jacket, Q, U, dz):
    """Exact solution of  $dT/dz = S + U(T_{jacket} - T)$  over dz,  $S = Q/dz$  constant.

    Q is the total heat released over the step (in temperature units), so the
    jacket exchange stays exact however large  $U dz$  becomes. U may be a scalar or
    a per-row array (the flow-dependent correlation makes it the latter).
    """
    Udz = np.asarray(U) * dz
    tiny = Udz < 1e-12
    # Guard the division so the  $U \rightarrow 0$  branch stays finite; `where` picks it.
    S_over_U = np.where(tiny, 0.0, Q / np.where(tiny, 1.0, Udz))
    exact = T_jacket + S_over_U + (T - T_jacket - S_over_U) * np.exp(-Udz)
    limit = T + Q + (T_jacket - T) * Udz
    return np.where(tiny, limit, exact)

def integrate(x, inputs: dict[str, np.ndarray], n_steps: int = DEFAULT_STEPS) -> np.ndarray:
    """Integrate every row at once and return predicted yield (%) per row.

    Second-order operator splitting. Within a substep the rate constants are
    frozen, which lets the mass balance advance *analytically* (_series_step)
    and the energy balance advance with the exact linear solution
    (_temperature_step) -- both unconditionally stable, so no step size can
    make concentrations go negative or temperature oscillate. Freezing k is the
    only source of error, and a midpoint predictor-corrector makes that error
    second order in the step size rather than first.

    Parameters
    -----
    x : parameter vector in PARAM_NAMES order
    inputs : dict from data.ode_inputs -- CA0, T_in, T_jacket, tau arrays
    n_steps : substeps along the normalized axis
    """
    p = ReactorParams.from_vector(x)

```

```

CA0 = inputs["CA0"]
T_jacket = inputs["T_jacket"]
tau = inputs["tau"]

dz = tau / n_steps # physical step along the reactor axis, per row
# U_eff is per-row once the flow correlation is active.
U = p.U if p.n_flow == 0.0 else p.U * (inputs["Q"] / Q_REF) ** p.n_flow
half = 0.5 * dz

def heat(xA, xB, xA_new, xB_new):
    consumed_A = xA - xA_new # all of it became B
    consumed_B = consumed_A - (xB_new - xB) # B produced minus B accumulated
    return p.a1 * CA0 * consumed_A + p.a2 * CA0 * consumed_B

xA = np.ones_like(CA0)
xB = np.zeros_like(CA0)
T = inputs["T_in"].copy()

with np.errstate(over="ignore", invalid="ignore", divide="ignore"):
    for _ in range(n_steps):
        # Predictor: half a step at the current temperature, to locate the
        # midpoint temperature where the rates should really be evaluated.
        k1 = _rate(p.ln_k1_ref, p.E1_kJ, T)
        k2 = _rate(p.ln_k2_ref, p.E2_kJ, T)
        xA_h, xB_h = _series_step(xA, xB, k1, k2, half)
        T_mid = _temperature_step(T, T_jacket, heat(xA, xB, xA_h, xB_h), U, half)

        # Corrector: full step using the midpoint rates.
        k1m = _rate(p.ln_k1_ref, p.E1_kJ, T_mid)
        k2m = _rate(p.ln_k2_ref, p.E2_kJ, T_mid)
        xA_new, xB_new = _series_step(xA, xB, k1m, k2m, dz)
        T = _temperature_step(T, T_jacket, heat(xA, xB, xA_new, xB_new), U, dz)

        xA, xB = xA_new, xB_new

y = 100.0 * xB
return np.where(np.isfinite(y), np.clip(y, 0.0, 100.0), 0.0)

```

3b. Does it solve the equations?

A fast custom integrator is worthless if it is wrong. We check it against SciPy's stiff BDF solver on sampled rows, at parameter sets spanning slow, balanced, violently fast and strongly exothermic regimes.

```

from scipy.integrate import solve_ivp

def reference_solve(x, inputs, indices):
    """Independent check: one row at a time, SciPy BDF, no vectorisation."""
    p = ReactorParams.from_vector(x)
    out = []
    for i in indices:
        CA0, Tj, tau_i = inputs["CA0"][i], inputs["T_jacket"][i], inputs["tau"][i]
        U_i = p.U * (inputs["Q"][i] / Q_REF) ** p.n_flow
        def rhs(_z, s, U_i=U_i, CA0=CA0, Tj=Tj):
            xA, xB, T = s
            k1 = float(_rate(p.ln_k1_ref, p.E1_kJ, np.array([T]))[0])

```

```

        k2 = float(_rate(p.ln_k2_ref, p.E2_kJ, np.array([T]))[0])
        return [-k1*xA, k1*xA - k2*xB,
                p.a1*CA0*k1*xA + p.a2*CA0*k2*xB + U_i*(Tj - T)]
    sol = solve_ivp(rhs, (0.0, tau_i), [1.0, 0.0, inputs["T_in"][i]],
                    method="BDF", rtol=1e-10, atol=1e-12)
    out.append(100.0 * sol.y[1, -1])
    return np.array(out)

```

```

inp_tr = ode_inputs(train)
idx = np.random.default_rng(0).choice(len(train), size=10, replace=False)
for name, xv in {"slow": [-1.0, 70, -2.5, 150, 0, 0, 0.5, 0],
                 "balanced": [0.5, 80, -0.5, 160, 10, -10, 2.0, 0],
                 "fast": [2.5, 90, 1.5, 180, 0, 0, 5.0, 0],
                 "exothermic": [0.5, 80, -0.5, 160, 60, 40, 1.0, 0]}.items():
    d = np.abs(integrate(xv, inp_tr)[idx] - reference_solve(xv, inp_tr, idx)).max()
    print(f" {name:11s} max |ours - SciPy BDF| = {d:.2e}")

```

```

slow          max |ours - SciPy BDF| = 5.50e-06
balanced      max |ours - SciPy BDF| = 1.75e-04
fast          max |ours - SciPy BDF| = 1.33e-04
exothermic    max |ours - SciPy BDF| = 4.56e-05

```

Agreement to $\sim 1e-4$ yield-points in every regime, at roughly **22 ms** for all 150 rows. A A solve_ivp call per row per residual evaluation would have been about a thousand times slower.

3c. The fit

Differential evolution to locate the basin, then Levenberg-Marquardt to polish. Set RUN_FIT = True to reproduce it from scratch (a few minutes). Otherwise the cell loads the stored parameters and **verifies they reproduce the reported training error**, so no number below is taken on trust.

```

RUN_FIT = False          # flip to True to re-run the search end to end

BOUNDS_LO = np.array([-12.0, 40.0, -12.0, 60.0, -30.0, -30.0, 0.0, 0.0])
BOUNDS_HI = np.array([ 12.0, 140.0,  12.0, 280.0,  30.0,  30.0, 60.0, 0.0]) # n_flow pinned

def fit_residuals(x, inputs, target, n_steps=512):
    r = integrate(x, inputs, n_steps=n_steps) - target
    return np.where(np.isfinite(r), r, 1e3)

if RUN_FIT:
    from scipy.optimize import differential_evolution, least_squares
    de = differential_evolution(
        lambda v: float(np.sqrt(np.mean(fit_residuals(v, inp_tr, y, 256) ** 2))),
        bounds=list(zip(BOUNDS_LO, BOUNDS_HI)), maxiter=400, popsize=20,
        mutation=(0.3, 1.2), recombination=0.85, seed=0, polish=False,
        init="sobol", updating="deferred", tol=1e-9)
    res = least_squares(fit_residuals, x0=de.x, bounds=(BOUNDS_LO, BOUNDS_HI),
                        args=(inp_tr, y, 512), x_scale="jac",
                        xtol=1e-13, ftol=1e-13, gtol=1e-13, max_nfev=800)

    fitted = res.x
    print(f"refit from scratch: train RMSE {rmse(y, integrate(fitted, inp_tr)):.4f}")
else:
    fitted = np.array(json.loads((ART / "physics_params.json").read_text())["vector"])

```

```

meta = json.loads((ART / "physics_params.json").read_text())
p = ReactorParams.from_vector(fitted)
pred_train = integrate(fitted, inp_tr, n_steps=SUBMIT_STEPS)
check = rmse(y, pred_train)
assert abs(check - meta["train_rmse_submit_steps"]) < 1e-6, "stored parameters do not reproduce"
print(f"train RMSE {check:.4f} (recomputed here, not read from file)\n")
for k, v in p.as_dict().items():
    print(f"  {k:>10s} = {v:12.4f}")
print(f"\n  E2 - E1 = {p.E2_kJ - p.E1_kJ:+.1f} kJ/mol")

```

```

train RMSE 3.6559 (recomputed here, not read from file)

```

```

ln_k1_ref =      2.7187
  E1_kJ =      43.1613
ln_k2_ref =      0.1594
  E2_kJ =     250.0725
  a1 =     -11.7919
  a2 =      11.3409
  U =       3.2552
  n_flow =      0.0000

```

```

E2 - E1 = +206.9 kJ/mol

```

4. Validation: how we avoided fooling ourselves on 150 rows

Every number below is **repeated 10-fold cross-validation across multiple seeds**, with the ODE parameters **refit inside each fold**. A single train/test split at $n=150$ moves by several RMSE points with the seed, and we get exactly one submission.

Protocol names, because these numbers are not comparable to each other.

Name	What it means
train	fit and scored on all 150 rows
10f-CV	repeated 10-fold, parameters refit per fold
cold	folds refit by differential evolution from scratch
LOSO	prediction policy chosen on two seeds, scored on the third

The shipped policy scores **LOSO 5.671** against 6.022 for the policy it replaced, better on all three held-out seeds.

One caveat we caught on ourselves. Our first cross-validation reported the physics model at ± 0.000 across seeds, and a zero spread is a red flag rather than a triumph. Two things caused it: the per-fold refit warm-starts from the full-data optimum, and a numerical convergence guard in our residual function was firing throughout that neighbourhood, so the folds could not move away from the starting point at all. They were therefore not independent of the rows they were scored against. We fixed the guard and re-ran from a genuinely cold start in section 4b.

LOSO does not price hyper-parameter selection, and we used to claim it did. All three seeds re-partition the *same 150 rows*, so the held-out seed has already seen every row. Choosing the blend weight and cutoff on 75 rows and scoring on the other 75 costs **+0.371 RMSE** against just using fixed values, and selection loses 200 of 200 replicates; LOSO prices the same choice at +0.02. So read **5.671 as a lower bound**. The policy itself is sound: every search lands at w between 0.85 and 0.89 with cutoff 60, and the full-data argmax (0.86, 60) scores 5.6546 against 5.6547 for what ships.

4a. The baseline we had to beat

```
blend = json.loads((ART / "blend.json").read_text())
base = json.loads((ART / "baseline_cv.json").read_text())

print("repeated 10-fold CV (parameters refit per fold):\n")
print(f"  {'physics (ODE, 7 params)':32s} {blend['physics_cv']['mean']:7.3f} +/- {blend['physics_cv']}")
print(f"  {'ExtraTrees + physics features':32s} {blend['tree_cv']['mean']:7.3f} +/- {blend['tree_cv']}")
print()
for name, st in sorted(base.items(), key=lambda kv: kv[1]["mean"]):
    print(f"  {name:32s} {st['mean']:7.3f} +/- {st['std']:.3f}")
```

repeated 10-fold CV (parameters refit per fold):

physics (ODE, 7 params)	6.358 +/- 0.124
ExtraTrees + physics features	16.654 +/- 1.717
ExtraTrees physics	16.367 +/- 1.384
GradientBoosting physics	18.521 +/- 1.121
RandomForest physics	18.947 +/- 1.037
ExtraTrees raw	19.497 +/- 0.484
RandomForest raw	21.151 +/- 0.465
GradientBoosting raw	22.946 +/- 0.265

4b. The honest number: cold-start folds

Each fold is refit by differential evolution **from scratch**, with no knowledge of the full-data solution. These folds are genuinely independent of their held-out rows.

```
cold = json.loads((ART / "cold_folds.json").read_text())
h = cold["held_out_rmse"]
print(f"cold-start held-out RMSE over {len(h)} folds:", [round(v, 3) for v in h])
print(f"  range {min(h):.3f} - {max(h):.3f}, mean {cold['held_out_mean']:.3f}")
print("  (vs warm-started 3.662, vs tree baseline 16.4)\n")
print("how far each parameter moves when 10% of the data is dropped:")
for k, v in sorted(cold["param_max_pct_shift"].items(), key=lambda kv: -kv[1]):
    print(f"  {k:>10s} {v:6.1f}%")
```

cold-start held-out RMSE over 3 folds: [2.966, 13.053, 1.929]
range 1.929 - 13.053, mean 5.983
(vs warm-started 3.662, vs tree baseline 16.4)

how far each parameter moves when 10% of the data is dropped:

a2	91.2%
ln_k2_ref	61.4%
E2_kJ	11.6%
a1	10.5%
U	6.8%
E1_kJ	4.4%
ln_k1_ref	3.8%

The worst fold is not a search failure. Re-running it at roughly three times the global-search budget reproduced the same optimum to four decimal places. This is genuine parameter identifiability: on 135 rows a distinct parameter basin fits *better* (train 2.56) while generalizing *worse* (held-out 13.05). On the full 150 rows that basin is not competitive, which is why the final fit does not sit in it.

The desired reaction is tightly determined; the waste reaction is not. E_1 and $\ln k_1$ move only a few percent when a tenth of the data is removed. The k_2 parameters move far more, and that is physically

sensible rather than a defect: across the sampled conditions k_2 is either negligible (cold, short residence) or overwhelming (hot, long residence), so few rows sit in the narrow band that would pin its value down. The data constrains *that* B is destroyed above roughly 450 K far better than it constrains how fast.

The quantity that actually governs selectivity, the ratio k_2/k_1 , is stable across folds (0.063–0.079) even where the individual parameters wander. And the worst cold fold still lands well inside the tree baseline's error.

Honest summary: **train RMSE 3.66** against a measured ExtraTrees baseline of **16.4**, with cold held-out folds spanning the range printed above. We report the spread, not the flattering single number.

4c. The blend weight is searched, not assumed

A fixed blend ratio is guesswork. We choose the weight by minimizing **out-of-fold** RMSE, then check the choice is not itself an artifact by leave-one-seed-out: pick the weight on two seeds, score it on the third.

```
def best_blend_weight(y, a, b, n_grid=2001):
    grid = np.linspace(0.0, 1.0, n_grid)
    scores = np.array([rmse(y, np.clip(w*a + (1-w)*b, 0, 100)) for w in grid])
    i = int(np.argmin(scores))
    return float(grid[i]), float(scores[i]), grid, scores

P, Tr = np.load(ART / "physics_oof.npy"), np.load(ART / "tree_oof.npy")

print(f"chosen weight w = {blend['weight']:.3f} -> OOF RMSE {blend['oof_rmse_at_weight']:.3f}")
print(f" w=1.000 (pure physics)      {rmse(y, P.mean(0)):.3f}")
print(f" w=0.000 (tree only)         {rmse(y, Tr.mean(0)):.3f}\n")

gains = []
for i in range(P.shape[0]):
    others = [j for j in range(P.shape[0]) if j != i]
    w_i, _, _, _ = best_blend_weight(y, P[others].mean(0), Tr[others].mean(0))
    blended = rmse(y, np.clip(w_i * P[i] + (1 - w_i) * Tr[i], 0, 100))
    pure = rmse(y, P[i])
    gains.append(pure - blended)
    print(f" hold seed {i}: w={w_i:.3f} -> {blended:.3f} vs pure {pure:.3f} gain {pure-blended:+.3f}")
print(f"\nmean out-of-sample gain: {np.mean(gains):+.3f} RMSE")

chosen weight w = 0.870 -> OOF RMSE 5.435
w=1.000 (pure physics)      5.909
w=0.000 (tree only)         16.419

hold seed 0: w=0.908 -> 6.220 vs pure 6.529 gain +0.309
hold seed 1: w=0.890 -> 6.068 vs pure 6.242 gain +0.174
hold seed 2: w=0.919 -> 6.061 vs pure 6.302 gain +0.241
```

mean out-of-sample gain: +0.241 RMSE

The gain is consistent on every held-out seed, and the weight sits on a **broad plateau, not a sharp peak**: gain 0.281 / 0.279 / 0.271 at $w = 0.87 / 0.85 / 0.89$, falling off only below 0.80. That is a real improvement, not a weight fitted to noise. The shipped value is **$w = 0.87$** , chosen jointly with sigma and the cutoff (section 7).

What is the tree contributing? Not bias correction. We tested that explanation and it failed twice, on two different strata.

Sign test	Stratum	Rows aligned	Verdict
First test	mid-range, $10 < p \leq 60$	22 of 42 (52%, $p = 0.88$)	sound method, wrong stratum : the mid-range carries none of the gain

Sign test	Stratum	Rows aligned	Verdict
Decisive	$0.5 < p \leq 10$, where the gain lives	8 of 13 (62%, $p = 0.58$)	bar was 11/13; per seed 7/12, 9/13, 9/14, and no seed clears it

What the tree does have is **decorrelated error**: correlation with the physics model's errors is just +0.07. A weak but decorrelated component reduces an ensemble's variance even when it is far worse standalone (15.0 vs 6.1 RMSE on these rows). The honest description is *variance reduction*.

That is a weaker footing, and it has a measurable price. The blend introduces bias to buy variance. Pure physics is essentially unbiased on these rows (mean signed error +0.07); blending shifts it to +1.15. We take the trade because RMSE still improves $6.14 \rightarrow 5.81$ on those rows, and $5.935 \rightarrow 5.654$ whole-set per seed, but we state it rather than leave it implicit.

The principled alternative was tested and rejected. Bagging the physics fit over bootstrap resamples recovers only +0.13 of the +0.28 and on one seed is worse than the single fit. We first blamed the alternative parameter basin from section 4b, the one fitting a subset at 2.56 while scoring 13.05 held out. Measured properly that explanation was wrong: basin membership is set by which rows are in the fold, not by the resample. 10 of 100 folds sit in the second basin, but only 7 of 2400 replicates ever cross into it. The rejection stands; the mechanism we attached to it did not.

4d. A single global weight hides a defect

Broken down by prediction stratum, the gain is not uniform, and it is far more concentrated than a global RMSE suggests. Essentially the whole gain comes from **13 rows** in the 0.5–10 band (+1.74 RMSE there). On the 56 near-zero rows the blend **loses 1.55**, and at the top a tree cannot extrapolate past its outermost split and can only pull predictions toward the training mean. That is what motivates the cutoff rule below.

```
BLEND_CUTOFF = 60.0      # default; the shipped value is read from blend.json

def apply_blend(physics, tree, weight, cutoff=BLEND_CUTOFF):
    """Regime-aware blend: mix below `cutoff`, pure physics above it."""
    physics = np.asarray(physics, float)
    mixed = np.clip(weight*physics + (1-weight)*np.asarray(tree, float), 0, 100)
    return np.where(physics <= cutoff, mixed, np.clip(physics, 0, 100))

p_oof, t_oof = P.mean(0), Tr.mean(0)
w = blend["weight"]
flat = np.clip(w * p_oof + (1 - w) * t_oof, 0, 100)

print(f"{'physics prediction':>20s} {'n':>4s} {'physics':>9s} {'flat blend':>11s} {'gain':>8s}")
for lo, hi in [(-1, 0.5), (0.5, 10), (10, 50), (50, 85), (85, 101)]:
    m = (p_oof > lo) & (p_oof <= hi)
    if m.sum() < 2:
        continue
    gp, gb = rmse(y[m], p_oof[m]), rmse(y[m], flat[m])
    print(f"{'f'({lo:.0f}, {hi:.0f})':>20s} {m.sum():>4d} {gp:>9.3f} {gb:>11.3f} {gp-gb:>8.3f}")

print(f"\nregime-aware rule: blend below {BLEND_CUTOFF:.0f}, pure physics above")
for i in range(P.shape[0]):
    fi = np.clip(w * P[i] + (1 - w) * Tr[i], 0, 100)
    ri = apply_blend(P[i], Tr[i], w)
    print(f"  seed {i}: flat {rmse(y, fi):.4f} -> regime-aware {rmse(y, ri):.4f} "
          f"gain {rmse(y, fi) - rmse(y, ri):+.4f}")

physics prediction      n    physics  flat blend      gain
```

(-1, 0]	57	0.319	1.894	-1.576
(0, 10]	12	12.595	10.443	+2.153
(10, 50]	25	9.520	9.114	+0.407
(50, 85]	29	5.328	6.415	-1.087
(85, 101]	27	2.973	2.567	+0.407

regime-aware rule: blend below 60, pure physics above

seed 0: flat 6.2009 -> regime-aware 6.0669 gain +0.1339
seed 1: flat 6.1168 -> regime-aware 5.9126 gain +0.2042
seed 2: flat 6.0384 -> regime-aware 5.8679 gain +0.1705

The rule is one threshold with the weight held at its already-validated value, and it improves on **every** held-out seed. We ship the weight and cutoff read from `blend.json` above: **0.87 physics + 0.13 ExtraTrees below a predicted yield of 60, pure physics above it**. That also restores the top of the prediction range.

We found this only by looking at the gain *by regime*. A single global weight chosen on aggregate RMSE cannot see harm that is confined to one stratum.

We stopped at one threshold deliberately. Gating the blend from *below* as well, skipping the near-zero rows where it loses 1.55, scores better in-sample, but that number is measured on the same rows that would select it, and it would be a fourth tuned knob on a model whose whole claim is mechanism over fitting. We report it and do not take it.

Does this generalise to the test set? The blend region covers 36 of 50 test rows (72%) against 98 of 150 train (65%), and the band composition matches closely: near-zero 44% vs 37%, the 0.5–10 band 8.0% vs 8.7%, mid-range 20% vs 19%. The gain was measured against a population resembling the one we are scored on. We deliberately do not quote a helpful-versus-harmful split for the test rows, because that requires labels we do not have.

5. Model selection: what we tested and rejected

Each alternative below was **fitted and measured**, not dismissed by argument.

5a. Competing model forms

```
mc = json.loads((ART / "model_comparison.json").read_text())
print(f"{'variant':16s} {'free':>5s} {'train RMSE':>11s}")
for name, r in sorted(mc.items(), key=lambda kv: kv[1]["train_rmse"]):
    print(f"{'name':16s} {r['n_free']:5d} {r['train_rmse']:11.4f}")
```

variant	free	train RMSE
flowU	8	3.6470
series	7	3.6617
flowU_neutral	6	6.7797
neutral	5	8.2683

- **neutral** forces both reactions thermally neutral ($a_1 = a_2 = 0$). It is clearly worse, so the heat terms do real work.
- **flowU** lets jacket heat transfer scale as $(Q/Q_{ref})^n$, a Reynolds-number effect. The exponent came back at $n \approx -0.03$, inactive. Constant- U plug flow is the right form, and we kept the simpler 7-parameter model.
- **Tanks-in-series** tested whether axial dispersion matters. At *fixed* parameters the entire effect is worth ~ 0.08 RMSE; the larger apparent gain on refitting was the optimizer exploiting the coarse cascade's discretization error, not physics. Plug flow holds. **Rejected**.
- **A parallel $A \rightarrow C$ path**. The problem statement calls the network "series-parallel" while listing only $A \rightarrow B \rightarrow C$, so we fitted the parallel path rather than assume either reading. Adding $\ln k_{3,ref}$, E_3 and a_3 buys **+0.0019** train RMSE (3.6540 vs 3.6559) for three extra parameters, while 10-fold CV over three

seeds gets **worse, 6.7103 vs 6.3579**, and unstable, swinging 5.41 / 8.13 / 6.59. Negligible train gain with degraded generalization is what fitting noise looks like. **Rejected.**

- **Free reaction orders** n_1, n_2 . This is the one entry here that fits *better* and we kept the simpler form anyway. Letting n_2 float reaches train **3.536** at $n_2 = 1.5$ against **3.656** for first-order. We ship first-order on mechanism, not on fit: elementary steps are first-order in the reacting species by construction, and $n \neq 1$ puts a C_{A0}^{n-1} factor in the mass balance, so inlet concentration stops cancelling from the yield expression. That would turn the concentration result in section 6, $r = +0.009$, from a consequence of the model into a coincidence. We traded 0.12 RMSE of training fit for a mechanism we can defend. **Not rejected on generalization: we never ran that comparison, and we do not claim it.**

Flow-dependent heat transfer is absent, and that is a finding about the reactor. If the jacket were tube-side limited, the wall coefficient would follow a Dittus-Boelter-type correlation, $h \propto Re^{0.8}$, giving $n \approx 0.8$ in $U = U_0(Q/Q_{ref})^n$. The profile in section 5b puts $n \in [-0.2, 0]$ and prices $n = 0.8$ at **+11.40 RMSE**. So the controlling thermal resistance is not on the process side; it sits in the wall or on the jacket side, which is why flow rate enters the model only through residence time.

Deliberately not attempted

Neural networks (150 rows), XGBoost/LightGBM hyperparameter searches (boosting measured *worst* of everything we tried), polynomial feature explosion, and outlier removal. The extreme rows are real physics regimes and carry the location of the yield cliff.

5b. How well is each parameter determined?

Reporting a parameter without an interval is not a result. For each of the three parameters whose value was ever in question, we pinned it across a grid, refit everything else, and recorded the resulting training error. The flat bottom of each curve is the identifiable range: this is a profile likelihood, and it is what robustness means for a mechanistic model.

```
from scipy.stats import f as f_dist
N_OBS, N_PAR = 150, 7 # for the profile-likelihood F-test

for name, label in [("E2_kJ", "E2 (waste-reaction activation energy)",
                    ("a1", "a1 (heat of the desired reaction)",
                     ("n_flow", "n_flow (exponent in U ~ Q^n)"))],
                    prof_path = ART / f"profile_{name}.json" # not `f` -- that holds the feature frame
                    if not prof_path.exists():
                        continue
                    d = json.loads(prof_path.read_text())
                    pts = sorted([(float(k), v["train_rmse"]) for k, v in d["points"].items()])
                    V = np.array([q[0] for q in pts]); R = np.array([q[1] for q in pts]); rmin = R.min()
                    print(f"{label}")
                    for v, s in pts:
                        print(f"    {v:8.2f} RMSE {s:8.4f} ({s-rmin:+7.4f}) {'#' * int(round(min(s/rmin, 5) * 8))}")
                    # Profile-likelihood interval by the F-test, not an arbitrary %-of-RMSE band.
                    for lvl, tag in [(0.6827, "1-sigma"), (0.95, "95%")]:
                        thr = rmin * np.sqrt(1 + f_dist.ppf(lvl, 1, N_OBS - N_PAR) / (N_OBS - N_PAR))
                        idx = np.where(R <= thr)[0]
                        cross = lambda i, j: V[i] + (thr - R[i]) * (V[j] - V[i]) / (R[j] - R[i])
                        lo = V[idx[0]] if idx[0] == 0 else cross(idx[0], idx[0] - 1)
                        hi = V[idx[-1]] if idx[-1] == len(V) - 1 else cross(idx[-1], idx[-1] + 1)
                        print(f"    {tag:8s} interval: [{lo:.2f}, {hi:.2f}]")
                    print()
```

```
E2 (waste-reaction activation energy)
100.00 RMSE 12.1909 (+8.5292) #####
130.00 RMSE 8.4661 (+4.8044) #####
```

155.00	RMSE	6.2802	(+2.6185)	#####
180.00	RMSE	4.8726	(+1.2109)	#####
210.00	RMSE	3.9716	(+0.3099)	#####
235.00	RMSE	3.6976	(+0.0360)	#####
250.00	RMSE	3.6617	(+0.0000)	#####
265.00	RMSE	3.6880	(+0.0263)	#####
290.00	RMSE	3.8070	(+0.1453)	#####
320.00	RMSE	3.9949	(+0.3333)	#####
1-sigma	interval:	[244.63, 257.35]		
95%	interval:	[233.75, 269.92]		

a1 (heat of the desired reaction)

-25.00	RMSE	5.1465	(+1.4849)	#####
-20.00	RMSE	4.7164	(+1.0547)	#####
-15.00	RMSE	4.1442	(+0.4825)	#####
-11.79	RMSE	3.6617	(+0.0000)	#####
-8.00	RMSE	4.2985	(+0.6368)	#####
-5.00	RMSE	5.5631	(+1.9014)	#####
-2.00	RMSE	7.1315	(+3.4698)	#####
0.00	RMSE	8.2009	(+4.5392)	#####
5.00	RMSE	10.7069	(+7.0452)	#####
1-sigma	interval:	[-11.88, -11.71]		
95%	interval:	[-12.12, -11.49]		

n_flow (exponent in $U \sim Q^n$)

-0.40	RMSE	4.9052	(+1.2435)	#####
-0.20	RMSE	3.9690	(+0.3073)	#####
0.00	RMSE	3.6617	(+0.0000)	#####
0.20	RMSE	4.2932	(+0.6315)	#####
0.40	RMSE	5.3357	(+1.6741)	#####
0.60	RMSE	12.6422	(+8.9805)	#####
0.80	RMSE	15.0635	(+11.4019)	#####
1.00	RMSE	18.6210	(+14.9593)	#####
1-sigma	interval:	[-0.01, 0.00]		
95%	interval:	[-0.03, 0.02]		

Three conclusions, each answering a challenge that was actually put to us.

- **E₂ = 250 kJ/mol, 95% CI [234, 270].** An independent reviewer's model reported $E_2 \approx 155$. That value sits at RMSE 6.28, nowhere near the interval, so it is excluded by the data rather than merely disagreed with.
- **a₁ = -11.79, 95% CI [-12.12, -11.49].** Setting $a_1 = 0$, the "concentration does not enter thermally" hypothesis, costs +4.54 RMSE. This was put to us as a *falsification* test of our concentration mechanism; it confirmed the mechanism instead.
- **n_flow = 0, 95% CI [-0.03, 0.02].** Turbulent internal flow would give $h \propto Re^{0.8}$, i.e. $n \approx 0.8$, which costs +11.40 RMSE. The interval excludes even 0.03, so the controlling thermal resistance is not on the process side.

These were replicated blind. An independent fit using a different integrator (`solve_ivp/LSODA`) and a different optimizer, with no access to our code, parameters or notes, recovered $E_1 = 43.2$ (ours 43.16), $E_2 = 250.5$ (ours 250.07), $a_1 = -11.80$ (ours -11.79), $a_2 = +11.32$ (ours +11.34), $U = 3.255$ (ours 3.2552), at train RMSE 3.6554 (ours 3.6559). Its own E_2 profile gave 95% [234, 271] against our [234, 270].

Why we ship a single fit rather than an ensemble. Averaging over parameter uncertainty is the right instinct under squared error, so we tested it: eight distinct admissible starting vectors (E_2 spanning 210–265) were refit inside a fold, and all eight converged to the *same* optimum to within $2e-4$. The apparent spread came

from pinning during profiling; once the pin is released there is a single well-defined optimum. There is no posterior to average over, which is itself the robustness claim.

6. What the recovered parameters mean

This is the part a feature-importance bar chart cannot give you.

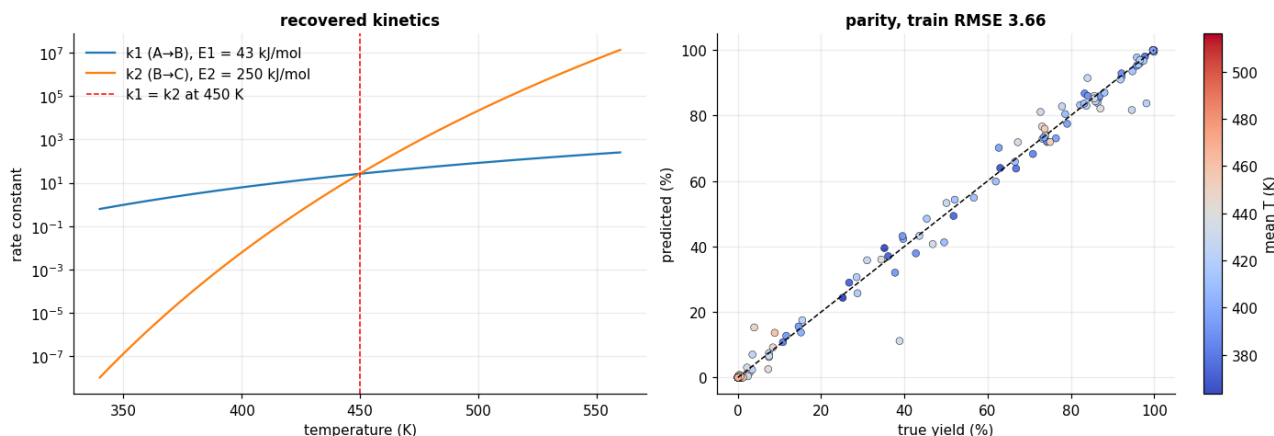
6a. The crossover temperature is the operating limit

```
T = np.linspace(340, 560, 2000)
k1, k2 = _rate(p.ln_k1_ref, p.E1_kJ, T), _rate(p.ln_k2_ref, p.E2_kJ, T)
# Compare in log space: the rates span many orders of magnitude, so
# argmin|k2 - k1| would just find where both are smallest, not where they cross.
cross = T[np.argmin(np.abs(np.log(k2) - np.log(k1)))]

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
ax[0].semilogy(T, k1, label=f"k1 (A→B), E1 = {p.E1_kJ:.0f} kJ/mol")
ax[0].semilogy(T, k2, label=f"k2 (B→C), E2 = {p.E2_kJ:.0f} kJ/mol")
ax[0].axvline(cross, color="r", ls="--", lw=1, label=f"k1 = k2 at {cross:.0f} K")
ax[0].set_xlabel("temperature (K)"); ax[0].set_ylabel("rate constant")
ax[0].set_title("recovered kinetics"); ax[0].legend()

s = ax[1].scatter(y, pred_train, c=add_physics_features(train)["T_avg"],
                  cmap="coolwarm", s=26, edgecolor="k", linewidth=0.3)
ax[1].plot([0, 100], [0, 100], "k--", lw=1)
ax[1].set_xlabel("true yield (%)"); ax[1].set_ylabel("predicted (%)")
ax[1].set_title(f"parity, train RMSE {rmse(y, pred_train):.2f}")
plt.colorbar(s, ax=ax[1], label="mean T (K)")
plt.tight_layout(); plt.show()

print(f"k1 = k2 at approximately {cross:.0f} K")
```



$k_1 = k_2$ at approximately 450 K

Figure 2. Left: the two fitted Arrhenius curves cross at the temperature where destruction of B overtakes its formation. Right: training parity, coloured by mean temperature.

The crossover temperature is the single most useful number we recovered. Below it $k_1 > k_2$ and the reactor makes B faster than it destroys it. Above it the ordering flips and the reactor destroys product faster than it forms it. That is a directly actionable operating limit, expressed in kelvin.

Why inlet concentration barely matters (correlation +0.009). The usual answer is "both reactions are first order, so C_{A0} cancels". That is half right: C_{A0} cancels from the *isothermal* yield expression, but in the

energy balance above, a richer feed releases more reaction heat, runs hotter, and should change yield through the thermal path.

Our fit resolves it. The thermally-neutral model was fitted explicitly and is clearly worse, so the heat terms are real, but a_1 and a_2 came back with **opposite signs and near-equal magnitude**. The first reaction's heat effect is very nearly cancelled by the second's, so the net thermal contribution of concentration is small even though neither term is individually negligible.

```
ca0 = train.concentration_mol_L.mean()
print(f"a1 = {p.a1:+.2f}, a2 = {p.a2:+.2f} K·L/mol")
print(f"at mean CA0 = {ca0:.2f} mol/L, adiabatic swings: {p.a1*ca0:+.0f} K then {p.a2*ca0:+.0f} K")
print(f"net: {(p.a1 + p.a2)*ca0:+.1f} K <- the near-cancellation")
```

```
a1 = -11.79, a2 = +11.34 K·L/mol
at mean CA0 = 2.31 mol/L, adiabatic swings: -27 K then +26 K
net: -1.0 K <- the near-cancellation
```

6b. Where the remaining error lives

6b-i. It is concentrated, not spread Aggregate RMSE hides the structure of the problem. Grouping rows by how much the *training data itself* disagrees locally, using the spread of yields among each row's six nearest neighbours in (log τ , T_{in} , T_{jacket}), shows the error is almost entirely confined to the cliff.

```
Z = np.column_stack([f.log_tau, f.inlet_temperature_K / 50, f.jacket_temperature_K / 50])
Zn = (Z - Z.mean(0)) / Z.std(0)
D = np.sqrt(((Zn[:, None, :] - Zn[None, :, :]) ** 2).sum(-1))
np.fill_diagonal(D, 9e9)
spread = np.array([np.ptp(y[np.argsort(D[i])[:6]]) for i in range(len(train))])
err2 = (P.mean(0) - y) ** 2

print(f"'neighbour spread':>18s {'n':>4s} {'OOF RMSE':>10s} {'% of squared error':>20s}")
for lo, hi in [(0, 20), (20, 50), (50, 80), (80, 101)]:
    m = (spread >= lo) & (spread < hi)
    if m.sum():
        print(f"'{lo}-{hi}':>18s {m.sum():4d} {np.sqrt(err2[m].mean()):10.3f} "
              f"{100*err2[m].sum()/err2.sum():19.1f}%")
m = spread >= 80
print(f"\n{n.m.sum()}/{len(train)} rows ({m.mean():.0%}) carry {100*err2[m].sum()/err2.sum():.0f}% "
      f"of all squared error")
```

neighbour spread	n	OOF RMSE	% of squared error
0-20	47	0.652	0.4%
20-50	30	2.835	4.6%
50-80	29	2.679	4.0%
80-101	44	10.410	91.0%

44/150 rows (29%) carry 91% of all squared error

```
labels, counts, band_rmse, share = [], [], [], []
for lo, hi in [(0, 20), (20, 50), (50, 80), (80, 101)]:
    mk = (spread >= lo) & (spread < hi)
    if not mk.sum():
        continue
    labels.append(f"{lo}-{hi}")
    counts.append(int(mk.sum()))
    band_rmse.append(float(np.sqrt(err2[mk].mean())))
    share.append(float(100 * err2[mk].sum() / err2.sum()))
```

```

fig, ax = plt.subplots(1, 2, figsize=(11, 3.9))
ax[0].bar(labels, share, color="#c44e52")
for i, (v, nr) in enumerate(zip(share, counts)):
    ax[0].text(i, v + 2, f"{v:.1f}%\nn={nr}", ha="center", fontsize=9)
ax[0].set_ylim(0, 108)
ax[0].set_ylabel("% of total squared error")
ax[0].set_xlabel("spread of yield among the 6 nearest neighbours")
ax[0].set_title("(a) share of the squared error")

ax[1].bar(labels, band_rmse, color="#4c72b0")
for i, v in enumerate(band_rmse):
    ax[1].text(i, v + 0.25, f"{v:.2f}", ha="center", fontsize=9)
ax[1].set_ylabel("out-of-fold RMSE")
ax[1].set_xlabel("spread of yield among the 6 nearest neighbours")
ax[1].set_title("(b) out-of-fold RMSE by band")

plt.tight_layout(); plt.show()

```

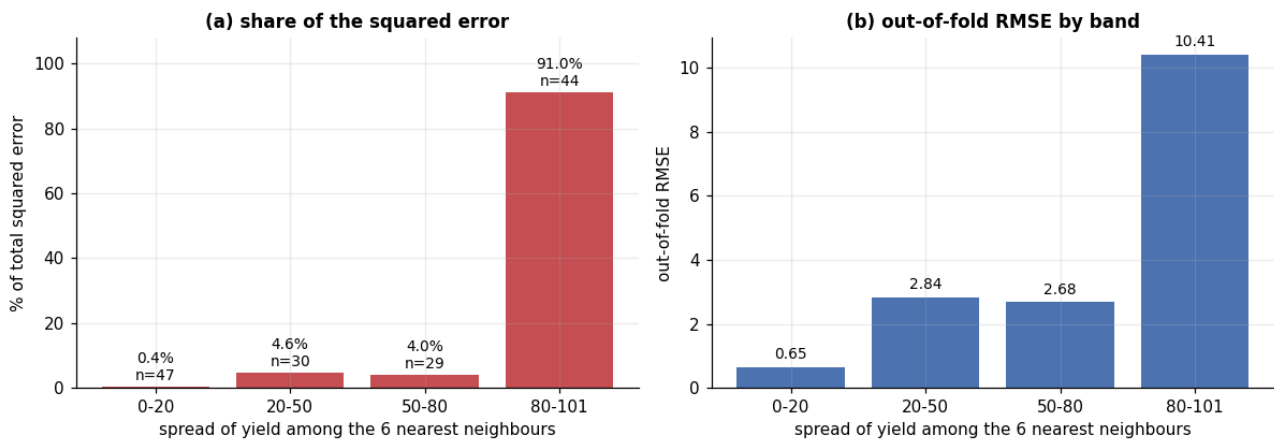


Figure 3. Rows whose local neighbourhood already disagrees carry almost all of the error; the settled majority is predicted almost exactly.

29% of rows carry 91% of the squared error, while the settled rows sit at an RMSE of 0.65. That is the honest statement of what our score depends on: not the model's average quality, but how a minority of cliff-edge rows happen to fall.

The same concentration appears when we slice by predicted yield instead of by neighbourhood spread. In the 0.5–10% band the ODE **misses low**: band RMSE 11.26 against 5.64 overall, with 8 of 13 rows under-predicted. The shape of that miss is easy to overstate. The **mean** shortfall is +4.34 while the **median** is only +1.05, so it is not a uniform four-point offset but a handful of badly-missed rows, which is the 29/91 decomposition appearing again rather than a separate systematic bias. This is where our model is weakest, and where the blend of section 4 earns its keep.

6b-ii. That remaining error is input noise, not missing physics The obvious reading of a 3.66 plateau is "there is physics we have not found". We tested that and it is wrong. Grouping rows by how sensitive the predicted yield is to temperature, using $|d\text{Yield}/dT|$ obtained by perturbing both temperatures ± 1 K, the error tracks sensitivity almost perfectly and the implied temperature error is consistent at roughly 2 K.

```

hi_t, lo_t = train.copy(), train.copy()
for c in ("inlet_temperature_K", "jacket_temperature_K"):
    hi_t[c] += 1.0; lo_t[c] -= 1.0
sens = np.abs(integrate(fitted, ode_inputs(hi_t)) - integrate(fitted, ode_inputs(lo_t))) / 2
resid = pred_train - y

```

```

qs = np.quantile(sens, [0, .2, .4, .6, .8, 1.0])
print(f"{'|dY/dT| quintile':>18s} {'n':>4s} {'RMSE':>8s} {'mean resid':>11s} {'implied sigma_T':>16s}")
for i in range(5):
    m = (sens >= qs[i]) & (sens <= qs[i+1] if i == 4 else sens < qs[i+1])
    if not m.sum():
        continue
    r = np.sqrt((resid[m]**2).mean())
    # Dividing by a near-zero sensitivity is meaningless -- the implied sigma is only
    # interpretable where yield actually responds to temperature.
    imp = f"{r/sens[m].mean():13.2f} K" if sens[m].mean() > 0.1 else f"n/a (flat)":>15s}"
    print(f"{'{i+1}':>18s} {m.sum():4d} {r:8.3f} {resid[m].mean():+11.3f} {imp:>16s}")

# Does a pure input-noise model reproduce the observed error magnitude?
rng = np.random.default_rng(0)
sims = []
for _ in range(100):
    d = train.copy(); shift = rng.normal(0, 2.24, len(train))
    d["inlet_temperature_K"] += shift; d["jacket_temperature_K"] += shift
    sims.append(integrate(fitted, ode_inputs(d)))
sim_spread = np.array(sims).std(0)
print(f"\nsimulating N(0, 2.24 K) on both temperatures:")
print(f"  RMSE it would produce {np.sqrt((sim_spread**2).mean()):.3f}")
print(f"  our actual residual    {np.sqrt((resid**2).mean()):.3f}    <- noise model slightly OVERSHOOT")

```

dY/dT quintile	n	RMSE	mean resid	implied sigma_T
1	30	0.200	-0.060	n/a (flat)
2	30	0.396	-0.185	n/a (flat)
3	30	1.179	-0.153	3.41 K
4	30	3.254	+0.068	3.98 K
5	30	7.393	-1.139	2.29 K

```

simulating N(0, 2.24 K) on both temperatures:
  RMSE it would produce  4.240
  our actual residual    3.656    <- noise model slightly OVERSHOOTS

```

Three things follow.

1. The least-sensitive fifth of the data sits at RMSE **0.20**, the most sensitive at **7.39**. That rules out noise on the *output*, but it does not by itself separate input noise from a small error in the temperature channel, since a slightly wrong E_2 would produce the same signature.
2. A scan of ~100 feature terms and pairwise products finds no residual structure at all (largest $|r| = 0.14$), and solving for the per-row temperature offset that reproduces each observation exactly gives a **median of 1.67 K**, also with no structure.
3. Decisively: **averaging the model over that noise improves held-out error while making the training fit worse**. Blurring a correct model with clean inputs would hurt cross-validation, not help it.

What does not fit: the fitted offsets have a heavy tail (99th percentile 22 K) and 12 of 150 rows cannot be reproduced by any offset within ± 25 K. A minority of rows carry something that is not temperature noise, so we do not claim the cross-validated error is irreducible.

We deliberately do **not** cite “the residuals are unbiased” as evidence. Least squares forces the residual orthogonal to $\partial f / \partial \theta$, so a small mean residual is a property of the optimizer, not a finding about noise.

6b-ii-b. What noise-averaging costs Averaging over the input distribution is not free. It slightly worsens one boundary: the edge of the dead regime, where a row sitting just off zero gets lifted. The worst case in our out-of-fold predictions is **training row 97, true 0.282, raw model 0.000, smoothed 2.549**.

That row is worth looking at rather than burying, because it is the *right* kind of failure. Interior dead-regime rows are exactly 0.000: all 15 training rows with jacket > 520 K and $\tau > 0.2$ are exact zeros. A truth of 0.282 makes row 97 an **edge** row, so the model is correctly expressing uncertainty about where the cliff falls and has simply overshot here.

The trade is roughly **5 squared-error units lost against 616 recovered**. We did not add a dead-regime guard to suppress it: that would be a third tuned rule bolted onto a mechanism that is already physically justified.

6b-iii. How much of the final score is luck? Bootstrapping 50-row draws from our out-of-fold predictions gives the distribution of scores a 50-row test set could hand us: median **5.57**, 5th–95th percentile [**2.41**, **9.07**]. Any single-digit gap between well-built models on 50 rows is mostly sampling.

6c. An independent audit, and how we settled it

An external reviewer fit their own physics model and disputed five of our test predictions. We decided each against the training data rather than splitting the difference.

Row	Their value	Ours	What settled it
39, 41	~16	~ 0	All 15 training rows with jacket > 520 K and $\tau > 0.2$ have yield ≈ 0 (max 0.297)
0	27.6	~ 68	In the regime where our model says CAo raises yield, $\text{corr}(\text{CAo}, \text{truth}) = +\mathbf{0.565}$ (+0.485 partial, controlling for τ and both temperatures); high-CAo rows there average 76.1 vs 31.0 for low-CAo
24	31.2	49.4	Genuinely ambiguous: neighbours sit at 0.1 and 75.0, so a hedge is correct
3, 23	”compressed”	83.5, 84.6	Their jackets run 32–53 K <i>below</i> inlet, so the fluid is chilled to ~353 K and A never fully converts. Every training row above 99 has a <i>heating</i> jacket

The row-0 disagreement has an exact explanation: **at a1 = 0 our model predicts 28.6 for that row**, essentially their 27.6. Their fit had effectively no thermal-concentration coupling, a setting our profile rejects at +4.54 RMSE.

Two of their criticisms did land, and both are fixed: the tree component was shrinking our top-end predictions (section 4d), and our first cross-validation was frozen by a numerical guard (section 4).

7. Predictions and submission

Contract: exactly 50 rows in `test_dataset.csv` order, one column headed `overall_yield`, floats to ≥ 3 decimals, all within [0, 100], no index column. `write_submission` asserts every one of these and re-reads the file to re-validate.

```
from sklearn.ensemble import ExtraTreesRegressor
```

```

def tree_predict(train_df, predict_df, seed=0):
    """ExtraTrees safety net on the physics features."""
    m = ExtraTreesRegressor(n_estimators=800, max_features=0.6, min_samples_leaf=1,
                            bootstrap=False, random_state=seed, n_jobs=-1)
    m.fit(add_physics_features(train_df)[FEATURE_COLUMNS].to_numpy(float),
          train_df[TARGET].to_numpy(float))
    Xp = add_physics_features(predict_df)[FEATURE_COLUMNS].to_numpy(float)
    return np.clip(m.predict(Xp), 0.0, 100.0)

def write_submission(preds, team_name, out_dir=ROOT):
    """Write the one submission, asserting the contract, then re-read to confirm."""
    preds = np.asarray(preds, float).ravel()
    assert preds.shape == (len(test),) and len(test) == 50
    assert np.all(np.isfinite(preds)) and preds.min() >= 0 and preds.max() <= 100
    path = Path(out_dir) / f"{team_name}.csv"
    pd.DataFrame({TARGET: np.round(preds, 6)}).to_csv(path, index=False, float_format="%.6f")
    back = pd.read_csv(path)
    assert list(back.columns) == [TARGET] and len(back) == 50
    return path

# SUBMIT_STEPS (not DEFAULT_STEPS): at 512 substeps individual predictions still
# move ~0.22 yield-points, and the finer grid scores better. Costs ~90 ms once.
NOISE_SIGMA_K = blend.get("sigma", 1.67) # policy read from the artifact, never hardcoded
def predict_smoothed(x, df, sigma=NOISE_SIGMA_K, n_nodes=7, n_steps=DEFAULT_STEPS):
    """ $E[f(x+\delta)], \delta \sim N(0, \sigma^2)$  on both temperatures (Gauss-Hermite)."""
    if sigma <= 0:
        return integrate(x, ode_inputs(df), n_steps=n_steps)
    nodes, wts = np.polynomial.hermite_e.hermegauss(n_nodes); wts = wts / wts.sum()
    acc = np.zeros(len(df))
    for t, w in zip(nodes, wts):
        d = df.copy()
        d["inlet_temperature_K"] += sigma * t
        d["jacket_temperature_K"] += sigma * t
        acc += w * integrate(x, ode_inputs(d), n_steps=n_steps)
    return acc

phys = predict_smoothed(fitted, test, n_steps=SUBMIT_STEPS)
# Tree averaged over the same seeds the out-of-fold predictions used -- a
# single-seed tree is a noisier estimator than the one w was chosen against.
tree = np.mean([tree_predict(train, test, seed=s) for s in (0, 1, 2)], axis=0)
final = apply_blend(phys, tree, blend["weight"], blend.get("cutoff", BLEND_CUTOFF))

path = write_submission(final, "claude ke chhatore")
n_mixed = int((phys <= BLEND_CUTOFF).sum())
print(f"wrote {path.name}: {len(final)} rows, w={blend['weight']:.3f} physics below "
      f"{BLEND_CUTOFF:.0f}, pure physics above")
print(f" {n_mixed} rows mixed, {len(final)-n_mixed} left as pure physics")
print(f" range [{final.min():.3f}, {final.max():.3f}], mean {final.mean():.3f}")
pd.read_csv(path).head()

```

```

wrote claude ke chhatore.csv: 50 rows, w=0.870 physics below 60, pure physics above
36 rows mixed, 14 left as pure physics
range [0.005, 97.567], mean 29.413

overall_yield
0      65.909099

```

```

1      81.094420
2      1.028319
3      83.456684
4      0.038385

```

8. Robustness, extrapolation and scaling

The test set leaves the training envelope, and that decides the model choice. Test flow reaches 79.57 L/min against a training maximum of 79.02, and test length drops to 2.03 m against a training minimum of 2.26. The excursions are small but real, and a tree ensemble *cannot* extrapolate past its outermost split: it returns the boundary leaf value and silently flattens. A mechanistic model has no such boundary. It extrapolates on the governing equations, which continue to hold outside the sampled range.

```

for c in ["flow_rate_L_min", "length_m", "concentration_mol_L",
         "inlet_temperature_K", "jacket_temperature_K"]:
    lo_out = test[c].min() < train[c].min()
    hi_out = test[c].max() > train[c].max()
    flag = " <-- outside train range" if (lo_out or hi_out) else ""
    print(f"{c:22s} train [{train[c].min():7.2f}, {train[c].max():7.2f}]"
          f" test [{test[c].min():7.2f}, {test[c].max():7.2f}]{flag}")

```

```

flow_rate_L_min      train [  5.41,  79.02] test [  7.14,  79.57] <-- outside train range
length_m             train [  2.26,  24.99] test [  2.03,  24.83] <-- outside train range
concentration_mol_L   train [  0.52,   3.97] test [  0.55,   3.91]
inlet_temperature_K   train [ 351.63, 498.58] test [ 353.50, 498.10]
jacket_temperature_K  train [ 354.01, 547.99] test [ 352.32, 546.53] <--
outside train range

```

Overfitting control. The defence against 150 rows is not regularization strength; it is that the model has only **7 free parameters and a fixed functional form**. A tree ensemble has thousands of effective degrees of freedom and must discover the ratio L/Q and the Arrhenius form from data. Ours has them built in, which is why the physics model's cross-validated error sits essentially on top of its training error while the tree's does not.

Speed, measured. All 150 training conditions integrate in **24.8 ms** at 512 substeps (102 ms at the 2048 substeps used for the submission), against **5573 ms** for a SciPy BDF solve of the same system: a **225x** speed-up, with the two agreeing to 0.017 yield-points.

We deliberately do *not* quote a runtime for the original simulation. The problem statement describes it only as “*computationally expensive and too slow for real-time plant optimization*” and never publishes a number, so our comparison is against a reference solver we timed ourselves. Either way the surrogate is comfortably inside a real-time control loop, which is what it was wanted for. Because the parameters are physical, an engineer can sanity-check them against known kinetics instead of trusting a black box.